Software Design Document (SDD)

CCRM - Centurion Customer Relationship Management System

Version: 1.5.0
Date: June 2, 2026
Architect: Development Team
Status: Final

---

1. DESIGN OVERVIEW

1.1 Architectural Style

Three-Tier Architecture:

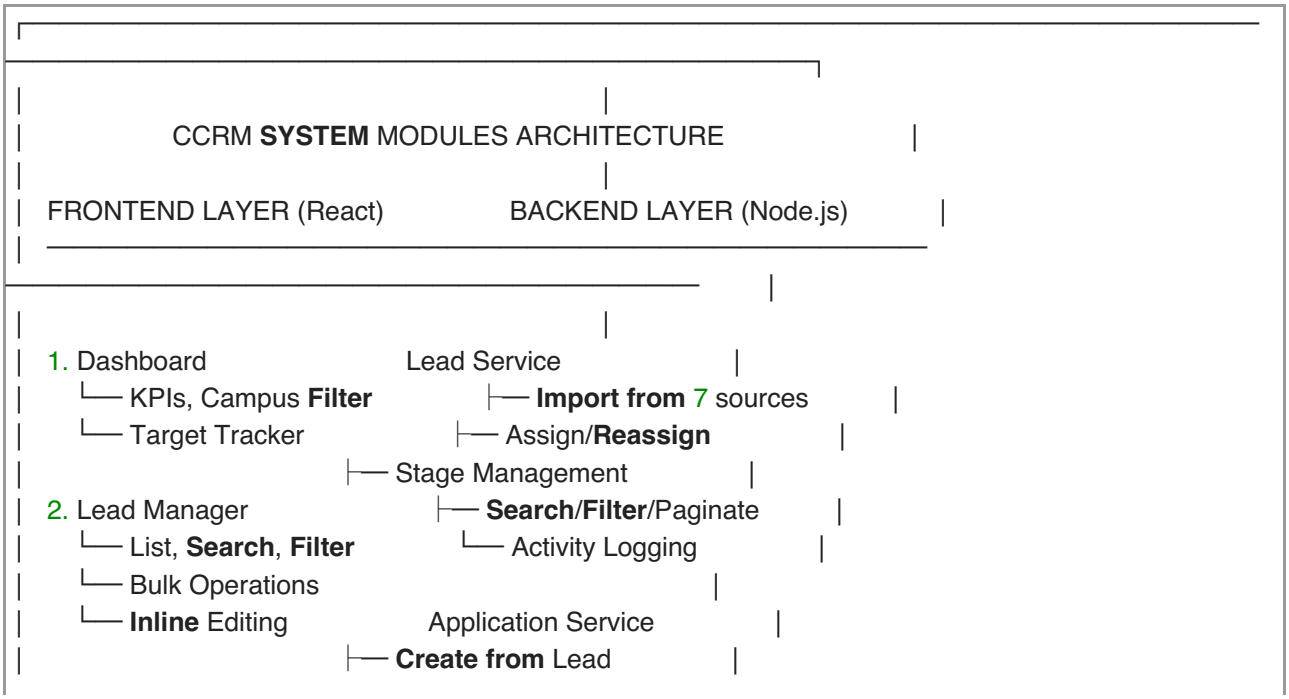
1. **Presentation Layer** — React frontend (Vite)
2. **Application Layer** — Node.js/Express backend
3. **Data Layer** — PostgreSQL database + AWS S3

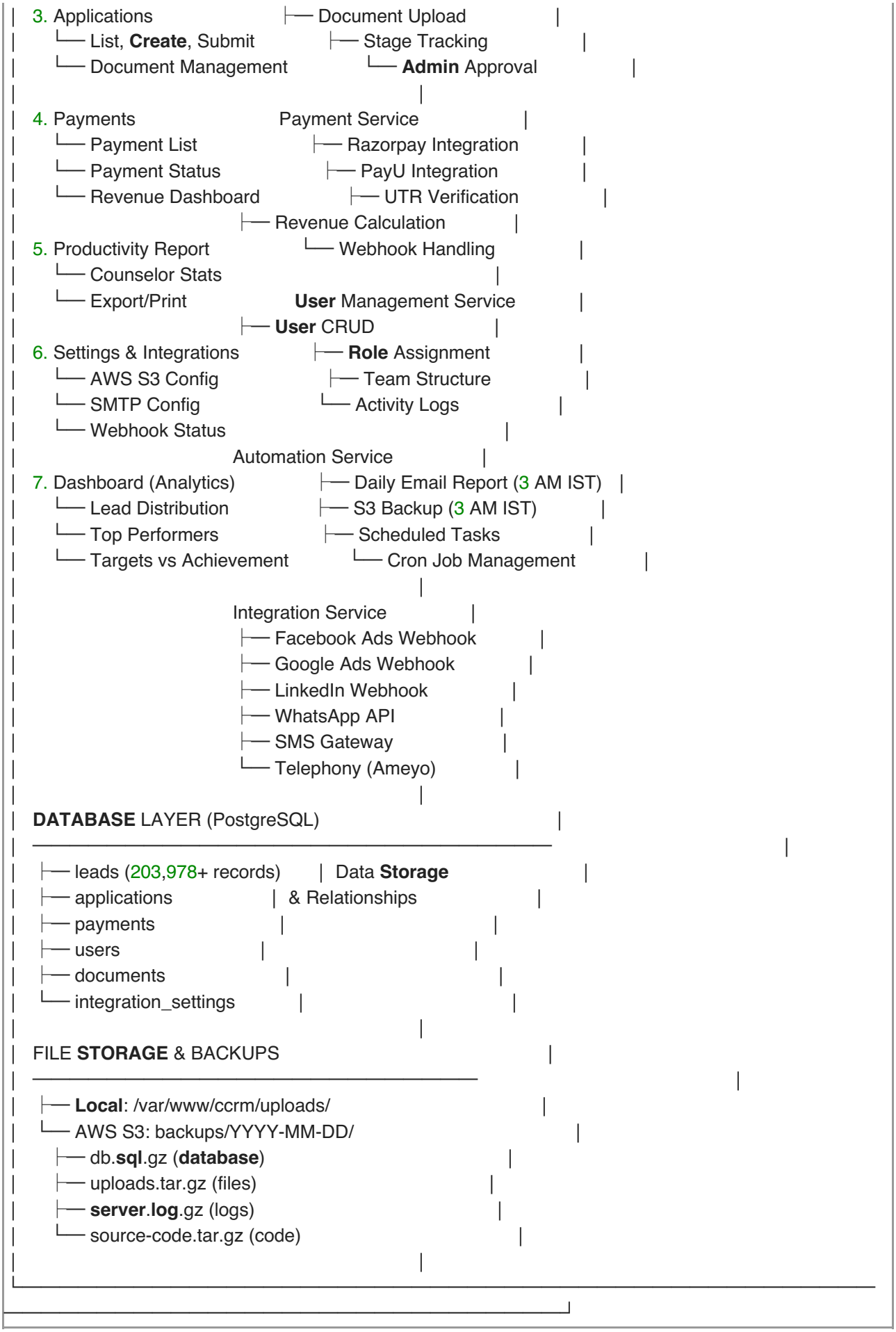
1.2 Key Design Principles

- **Scalability:** Server-side pagination, role-scoped queries
- **Security:** RBAC, JWT tokens, input validation
- **Maintainability:** Modular code, clear separation of concerns
- **Performance:** Indexed database queries, optimized API responses
- **Reliability:** Error handling, transaction consistency, automated backups

1.3 System Modules Architecture (Quick Reference)

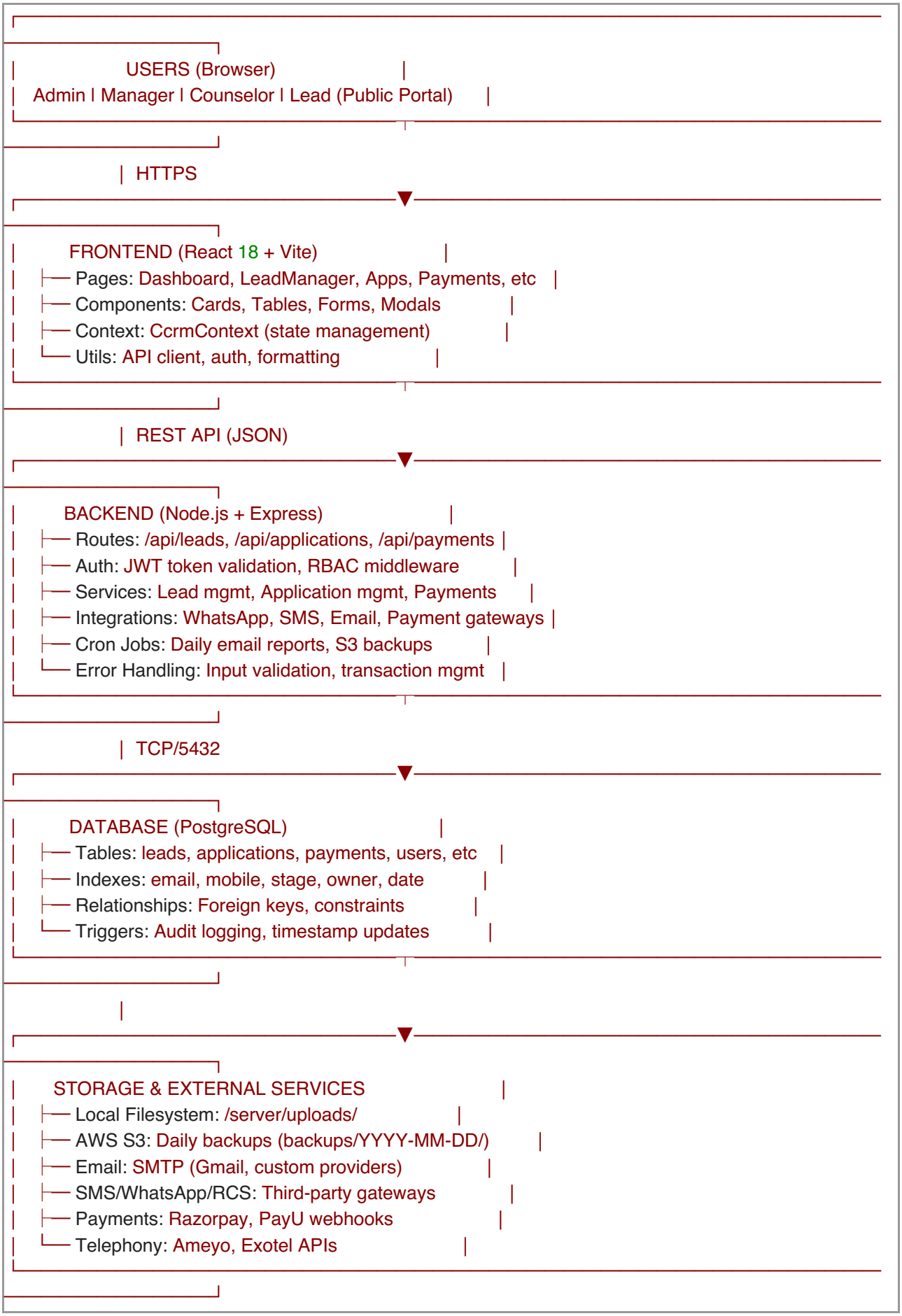
Core Modules & Their Interactions:




3. Applications
├── Document Upload
└── List, Create, Submit
├── Stage Tracking
└── Document Management
└── Admin Approval
4. Payments
Payment Service
└── Payment List
├── Razorpay Integration
└── Payment Status
├── PayU Integration
└── Revenue Dashboard
├── UTR Verification
├── Revenue Calculation
5. Productivity Report
└── Webhook Handling
└── Counselor Stats
└── Export/Print
User Management Service
├── User CRUD
6. Settings & Integrations
├── Role Assignment
└── AWS S3 Config
├── Team Structure
└── SMTP Config
└── Activity Logs
└── Webhook Status
Automation Service
7. Dashboard (Analytics)
├── Daily Email Report (3 AM IST)
└── Lead Distribution
├── S3 Backup (3 AM IST)
└── Top Performers
├── Scheduled Tasks
└── Targets vs Achievement
└── Cron Job Management
Integration Service
├── Facebook Ads Webhook
├── Google Ads Webhook
├── LinkedIn Webhook
├── WhatsApp API
├── SMS Gateway
└── Telephony (Ameyo)
DATABASE LAYER (PostgreSQL)
├── leads (203,978+ records)
├── applications
├── payments
├── users
├── documents
└── integration_settings
Data Storage
& Relationships
FILE STORAGE & BACKUPS
├── Local: /var/www/ccrm/uploads/
└── AWS S3: backups/YYYY-MM-DD/
├── db.sql.gz (database)
├── uploads.tar.gz (files)
├── server.log.gz (logs)
└── source-code.tar.gz (code)

2. SYSTEM ARCHITECTURE

2.1 High-Level Architecture Diagram


USERS (Browser)
Admin | Manager | Counselor | Lead (Public Portal)
HTTPS
FRONTEND (React 18 + Vite)
├── Pages: Dashboard, LeadManager, Apps, Payments, etc
├── Components: Cards, Tables, Forms, Modals
├── Context: CcrmContext (state management)
└── Utils: API client, auth, formatting
REST API (JSON)
BACKEND (Node.js + Express)
├── Routes: /api/leads, /api/applications, /api/payments
├── Auth: JWT token validation, RBAC middleware
├── Services: Lead mgmt, Application mgmt, Payments
├── Integrations: WhatsApp, SMS, Email, Payment gateways
├── Cron Jobs: Daily email reports, S3 backups
└── Error Handling: Input validation, transaction mgmt
TCP/5432
DATABASE (PostgreSQL)
├── Tables: leads, applications, payments, users, etc
├── Indexes: email, mobile, stage, owner, date
├── Relationships: Foreign keys, constraints
└── Triggers: Audit logging, timestamp updates
STORAGE & EXTERNAL SERVICES
├── Local Filesystem: /server/uploads/
├── AWS S3: Daily backups (backups/YYYY-MM-DD/)
├── Email: SMTP (Gmail, custom providers)
├── SMS/WhatsApp/RCS: Third-party gateways
├── Payments: Razorpay, PayU webhooks
└── Telephony: Ameyo, Exotel APIs

2.2 Technology Stack

Layer	Technology	Version	Purpose
Frontend	React	18+	UI framework
	Vite	5.4+	Build tool
	Lucide Icons	-	UI icons
	TailwindCSS	3.3+	Styling
Backend	Node.js	18+	Runtime
	Express	4.18+	Web framework
	PostgreSQL	12+	Database
	node-cron	3.0+	Job scheduling
	Nodemailer	6.10+	Email sending
	AWS SDK	3.600+	S3 operations
	Multer	1.4+	File uploads
	JWT	9.0+	Authentication
	bcryptjs	2.4+	Password hashing
Infrastructure	Ubuntu	20.04+	OS
	systemd	-	Service management
	AWS S3	-	Backup storage

---

3. DATABASE DESIGN

3.1 Entity-Relationship Diagram

```
users
 ├── id (PK)
 ├── email (UNIQUE)
 ├── password (hashed)
 ├── name
 ├── role (Admin|Manager|Counselor|Lead)
 ├── status (Active|Inactive)
 ├── campus
 ├── created_at
 └── updated_at

leads
 ├── id (PK)
 ├── email (INDEX)
 ├── mobile (INDEX)
 ├── name
 ├── campus
 ├── course
 ├── stage (INDEX) — Untouched|Contacted|...|Converted
 ├── owner (FK→users, INDEX) — counselor assigned
 ├── source — Facebook|Google|Instagram|LinkedIn|Manual
 ├── lead_details (JSONB) — custom fields, campaign data
 ├── created_at (INDEX)
 ├── updated_at
```

```
└── deleted_at (soft delete)

applications
├── id (PK)
├── app_no (UNIQUE) — APP-XXXX-YYYY
├── lead_id (FK→leads, INDEX)
├── owner (FK→users) — counselor who created
├── stage — Draft|Submitted|Approved|Rejected|Enrolled
├── campus
├── course
├── created_at
└── updated_at

payments
├── id (PK)
├── app_no (FK→applications, INDEX)
├── lead_id (FK→leads, INDEX)
├── amount (in paise, e.g., 50000 = ₹500)
├── status — Pending|Approved|Paid|Failed|Refunded
├── utr_number (UNIQUE, INDEX) — Unique Transaction Reference
├── payment_method — Online|Manual
├── gateway — Razorpay|PayU|Manual
├── verified_by (FK→users) — admin who verified
├── verified_at
├── created_at
└── updated_at

documents
├── id (PK)
├── app_id (FK→applications, INDEX)
├── file_name
├── file_path — /uploads/documents/APP-XXXX/filename
├── file_size
├── file_type — PDF|JPG|PNG
├── uploaded_by (FK→users)
├── created_at
└── updated_at

integration_settings
├── id (PK)
├── key (UNIQUE, INDEX) — smtp_host, aws_access_key_id, etc.
├── value (encrypted) — secret values
├── updated_at
└── created_at

activity_logs
├── id (PK)
├── user_id (FK→users, INDEX)
├── entity_type — Lead|Application|Payment
├── entity_id (INDEX)
├── action — Create|Update|Delete|Assign
├── old_value (JSONB)
```

```
├── new_value (JSONB)
├── created_at (INDEX)
└── metadata (JSONB)

targets
├── id (PK)
├── month
├── year
├── campus
├── target_leads
├── target_applications
├── target_enrollments
└── created_at
```

3.2 Database Indexes

```
CREATE INDEX idx_leads_email ON leads(email);
CREATE INDEX idx_leads_mobile ON leads(mobile);
CREATE INDEX idx_leads_owner ON leads(owner);
CREATE INDEX idx_leads_stage ON leads(stage);
CREATE INDEX idx_leads_created_at ON leads(created_at DESC);

CREATE INDEX idx_applications_app_no ON applications(app_no);
CREATE INDEX idx_applications_lead_id ON applications(lead_id);
CREATE INDEX idx_applications_owner ON applications(owner);

CREATE INDEX idx_payments_app_no ON payments(app_no);
CREATE INDEX idx_payments_lead_id ON payments(lead_id);
CREATE INDEX idx_payments_utr_number ON payments(utr_number);
CREATE INDEX idx_payments_status ON payments(status);

CREATE INDEX idx_activity_logs_user_id ON activity_logs(user_id);
CREATE INDEX idx_activity_logs_entity ON activity_logs(entity_type, entity_id);
CREATE INDEX idx_activity_logs_created_at ON activity_logs(created_at DESC);

CREATE INDEX idx_integration_settings_key ON integration_settings(key);
```

3.3 Key Constraints & Relationships

```
-- Foreign Keys
ALTER TABLE leads
ADD CONSTRAINT fk_leads_owner FOREIGN KEY (owner) REFERENCES users(name);

ALTER TABLE applications
ADD CONSTRAINT fk_applications_lead FOREIGN KEY (lead_id) REFERENCES leads(id) ON DELETE
CASCADE;

ALTER TABLE payments
ADD CONSTRAINT fk_payments_app FOREIGN KEY (app_no) REFERENCES applications(app_no);

-- Unique Constraints
ALTER TABLE users ADD CONSTRAINT uq_email UNIQUE (email);
ALTER TABLE applications ADD CONSTRAINT uq_app_no UNIQUE (app_no);
ALTER TABLE payments ADD CONSTRAINT uq_utr UNIQUE (utr_number);

-- Check Constraints
ALTER TABLE leads
ADD CONSTRAINT chk_stage CHECK (stage IN ('Untouched', 'Contacted', 'Follow Up', 'Interested',
'Process for Payment', 'Payment Success', 'Converted'));
```

---

4. API DESIGN

4.1 API Architecture

Base URL: https://crm.cutmap.ac.in/api

Authentication: JWT Bearer token in Authorization header

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

4.2 API Endpoints (Core)

Authentication

```
POST   /auth/login          — Login with email/password
POST   /auth/logout         — Logout and invalidate token
POST   /auth/refresh        — Refresh JWT token
POST   /auth/reset-password — Password reset via email
```

Leads

```
GET    /leads               — List leads (paginated, role-scoped)
GET    /leads/:id           — Get single lead details
POST   /leads               — Create lead (admin/manager)
PUT    /leads/:id           — Update lead
DELETE /leads/:id           — Soft delete lead
PUT    /leads/:id/stage     — Update lead stage
PUT    /leads/:id/owner     — Assign lead to counselor
POST   /leads/bulk/upload   — Bulk import leads (Excel)
POST   /leads/bulk/assign   — Bulk assign leads
```

Applications

```
GET    /applications       — List applications (paginated)
GET    /applications/:id   — Get single application
POST   /applications       — Create application
PUT    /applications/:id   — Update application
PUT    /applications/:id/stage — Update application stage
GET    /applications/:app_no/documents — List documents
POST   /applications/:app_no/documents — Upload document
DELETE /applications/:app_no/documents/:id — Delete document
```

Payments

```
GET    /payments           — List payments
GET    /payments/:id       — Get single payment
POST   /payments           — Record payment
PUT    /payments/:id       — Update payment
POST   /payments/:id/verify — Admin verify with UTR
POST   /payments/webhook/razorpay — Razorpay webhook
POST   /payments/webhook/payu — PayU webhook
```

Users & Teams

```
GET    /users              — List users (admin only)
POST   /users              — Create user (admin only)
PUT    /users/:id          — Edit user (admin only)
DELETE /users/:id          — Deactivate user (admin only)
GET    /users/:id/activity — User's recent actions
POST   /users/bulk/import  — Bulk import users (Excel)
GET    /teams/:manager_id  — List team members
```

Analytics & Reports

```
GET    /dashboard/stats    — KPI summary (role-scoped)
GET    /campus/stats       — Campus-wise statistics
GET    /users/:user_id/stats — Individual counselor stats
GET    /targets/achievement — Month-wise achievement
POST   /targets            — Set admission targets
GET    /activity-logs      — Audit trail (admin only)
```

Integrations

```
GET    /integration-settings — List all configured integrations
POST   /integrations       — Save integration settings
POST   /admin/test-daily-report — Trigger email + backup (admin)
```

4.3 Request/Response Format

Request:

```
{
  "method": "GET",
  "url": "https://crm.cutmap.ac.in/api/leads?page=1&limit=50&owner=John%20Doe",
  "headers": {
    "Authorization": "Bearer <token>",
    "Content-Type": "application/json"
  }
}
```

Response (Success):

```
{
  "status": "success",
  "data": {
    "leads": [
      {
        "id": 1,
        "name": "Rahul Kumar",
        "email": "rahul@gmail.com",
        "mobile": "9876543210",
        "stage": "Interested",
        "owner": "John Doe",
        "created_at": "2026-06-01T10:30:00Z"
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 50,
      "total": 203978,
      "total_pages": 4080
    }
  }
}
```

Response (Error):

```
{
  "status": "error",
  "error": "Unauthorized",
  "code": 401,
  "message": "Invalid or expired token"
}
```

4.4 Pagination Strategy

Server-Side Pagination:

- Default limit: 50 records per request
- Maximum limit: 500 records per request
- Offset-based: ?page=1&limit=50 translates to OFFSET 0 LIMIT 50
- Prevents loading 200k+ records into browser memory

SQL Query Pattern:

```
SELECT id, name, email, mobile, stage, owner, created_at
FROM leads
WHERE owner = $1 AND deleted_at IS NULL
ORDER BY created_at DESC
LIMIT 50 OFFSET 0;
```

4.5 Error Handling & Status Codes

Code	Scenario
200	OK — Request succeeded
201	Created — Resource created
400	Bad Request — Invalid input
401	Unauthorized — Missing/invalid token
403	Forbidden — Insufficient permissions
404	Not Found — Resource doesn't exist
409	Conflict — Duplicate entry
500	Server Error — Internal error
503	Service Unavailable — Maintenance

---

5. FRONTEND ARCHITECTURE

5.1 Project Structure

```
ccrm/
├── src/
│   ├── pages/
│   │   ├── Dashboard.jsx          — KPI dashboard
│   │   ├── LeadManager.jsx        — Lead list + management
│   │   ├── Applications.jsx       — Application list
│   │   ├── Payments.jsx           — Payment tracking
│   │   ├── ProductivityReport.jsx — Counselor stats
│   │   ├── Users.jsx              — User management
│   │   ├── Integrations.jsx       — Settings for integrations
│   │   ├── IntegrationHealth.jsx  — Integration status
│   │   ├── Settings.jsx           — General settings
│   │   ├── Analytics.jsx          — Advanced analytics
│   │   └── Login.jsx              — Authentication
│   │
│   ├── components/
│   │   ├── Sidebar.jsx            — Navigation sidebar
│   │   ├── Header.jsx             — Top navigation
│   │   ├── Card.jsx               — Reusable card component
│   │   ├── Table.jsx              — Reusable table component
│   │   ├── Modal.jsx              — Modal dialog
│   │   ├── Form.jsx               — Form inputs
│   │   ├── Loader.jsx             — Loading indicator
│   │   └── IntegrationStatusWidget.jsx — Integration status display
│   │
│   ├── context/
│   │   └── CcrmContext.jsx        — Global state (user, campus, settings)
│   │
│   ├── utils/
│   │   ├── api.js                 — API client (fetch wrapper)
│   │   ├── auth.js                — Token management
│   │   ├── formatting.js          — Format numbers, dates, etc.
│   │   └── constants.js           — Enums, magic numbers
│   │
│   ├── App.jsx                    — Main app component
│   ├── version.js                 — App version
│   └── main.jsx                   — Entry point
│
├── public/
│   ├── index.html
│   ├── favicon.ico
│   └── manifest.json
│
├── vite.config.js                 — Vite build configuration
├── tailwind.config.js             — TailwindCSS setup
└── package.json
```

5.2 Component Architecture

Page Components:

- Fetch data from API on mount

- Manage page-level state
- Handle user interactions
- Emit actions to context when needed

Reusable Components:

- Pure functional components (no side effects)
- Accept props for data and callbacks
- No API calls (handled by pages)
- Focused on presentation

Context Usage:

- Global state: currentUser, activeCampus, showToast
- Auth token storage
- User profile and permissions
- Toast notifications

5.3 State Management

```
// CcrmContext provides:
{
  // Auth & User
  currentUser: { id, email, name, role },
  isLoggedIn: boolean,
  logout: () => void,

  // UI State
  activeCampus: string,
  setActiveCampus: (campus) => void,
  counselors: array,
  managers: array,

  // Notifications
  showToast: (message, type) => void,

  // Actions
  saveTarget: (targetForm) => Promise,
  refreshData: () => Promise
}
```

5.4 Key Pages & Flows

Dashboard:

- Displays KPIs (Total Leads, Applications, Revenue, Enrollments)
- Campus filter buttons
- Target vs. achievement tracker
- Load time: <3 seconds
- Role-scoped: Counselor sees only own stats, Manager sees team stats, Admin sees all

Lead Manager:

- Paginated lead list (50 per page)

- Search/filter by name, email, mobile, stage, owner
- Inline name editing with pencil icon
- Bulk operations (assign, delete)
- Lead count respects role scoping

Productivity Report:

- Counselor-wise statistics table
- Quick Summary view (Lead Assigned, Untouched, Interested, etc.)
- Exportable as CSV
- Pagination (6 rows per page)

---

6. BACKEND ARCHITECTURE

6.1 Server Structure

```
server/
├── index.js              — Main server file
├── db.js                 — Database connection pool
├── package.json          — Dependencies
├── .env                  — Environment variables
├── uploads/              — Uploaded files
│   ├── documents/        — Application documents
│   └── avatars/          — User avatars
└── logs/
    └── server.log        — Application logs
```

6.2 Middleware Stack

```
// Global Middleware (applied to all requests)
app.use(cors())                          // CORS headers
app.use(express.json())                  // JSON parsing
app.use(express.static(distPath))        // Static frontend
app.use(errorHandler)                    // Error handling

// Route-level Middleware
app.get('/api/...',   authenticateToken, handler) // Auth required
app.post('/api/...',  authenticateToken, handler)
app.put('/api/...',   authenticateToken, handler)
app.delete('/api/...', authenticateToken, roleCheck('Admin'), handler) // Admin only
```

6.3 Request Flow

```
Request
  ↓
CORS Middleware (add headers)
  ↓
Body Parser (parse JSON)
  ↓
Route Matching (GET /api/leads)
  ↓
Authenticate Token (verify JWT)
  ↓
Role Check (admin? manager? counselor?)
  ↓
Handler Logic
  ├── Validate input
  ├── Query database
  ├── Transform response
  └── Return JSON
  ↓
Error Handler (catch exceptions)
  ↓
Response (200/400/500 + JSON)
```

6.4 Key Functions

Authentication

```
// Generate JWT on login
const token = jwt.sign({ id, email, role }, SECRET, { expiresIn: '24h' })

// Verify token on each request
app.use((req, res, next) => {
  const token = req.headers['authorization']?.split(' ')[1]
  try {
    req.user = jwt.verify(token, SECRET)
    next()
  } catch (e) {
    res.status(401).json({ error: 'Unauthorized' })
  }
})
```

Role Scoping

```
// Counselor sees only own leads
if (req.user.role === 'Counselor') {
  query += ` AND owner = $1`
  params.push(req.user.name)
}

// Manager sees team leads
if (req.user.role === 'Manager') {
  const team = await getTeamMembers(req.user.name)
  query += ` AND owner IN (${team.map((_, i) => `$${i+1}`).join(',')})`
  params.push(...team)
}

// Admin sees all leads (no WHERE clause)
```

Pagination

```
const page = Math.max(1, parseInt(req.query.page) || 1)
const limit = Math.min(500, parseInt(req.query.limit) || 50)
const offset = (page - 1) * limit

const result = await pool.query(
  `SELECT * FROM leads WHERE ... LIMIT $1 OFFSET $2`,
  [limit, offset]
)

res.json({
  data: result.rows,
  pagination: { page, limit, total: result.rowCount }
})
```

Daily Email Report (Cron)

```
async function sendProductivityEmailReport() {
  const stats = await getStatsFromDB()
  const recipients = await getIntegrationSetting('report_email_recipients')
  const html = buildHtmlTable(stats)

  const transporter = nodemailer.createTransport(smtpConfig)
  await transporter.sendMail({
    from: config.from,
    to: recipients,
    subject: `Productivity Report — ${new Date().toLocaleDateString()}`,
    html: html
  })
}

cron.schedule('0 3 * * *', sendProductivityEmailReport, {
  timezone: 'Asia/Kolkata'
})
```

S3 Backup (Cron)

```
async function performS3Backup() {
  const s3 = new S3Client({ credentials, region })
  const dateStr = new Date().toISOString().split('T')[0]

  // Database dump
  const db = await execAsync(`pg_dump ... | gzip`)
  await s3.send(new PutObjectCommand({
    Bucket: bucket,
    Key: `backups/${dateStr}/db.sql.gz`,
    Body: db
  }))

  // Uploads tar
  const uploads = await execAsync(`tar -czf - uploads/`)
  await s3.send(new PutObjectCommand({
    Bucket: bucket,
    Key: `backups/${dateStr}/uploads.tar.gz`,
    Body: uploads
  }))

  // Logs
  const logs = await execAsync(`journalctl ... | gzip`)
  await s3.send(new PutObjectCommand({
    Bucket: bucket,
    Key: `backups/${dateStr}/server.log.gz`,
    Body: logs
  }))
}
```

---

7. SECURITY DESIGN

7.1 Authentication & Authorization

```
Login
  ↓
Email + Password → Verify against bcrypt hash
  ↓
Generate JWT token (24h expiry)
  ↓
Store in localStorage on frontend
  ↓
Each API request includes token in header
  ↓
Server verifies JWT signature
  ↓
Extract user role from token
  ↓
Check role against endpoint requirements
  ↓
Allow/deny request
```

7.2 Password Security

```
// On registration/password change:
const hashedPassword = await bcrypt.hash(plainPassword, 10)
// Store hashedPassword in database (never plain text)

// On login:
const isValid = await bcrypt.compare(plainPassword, hashedPassword)
```

7.3 Data Protection

```
// Sensitive fields are masked in API responses
const response = {
  id: 1,
  email: 'user@example.com',
  // secret: '****' (not sent)
  // password: '****' (not sent)
}

// Credentials stored in database, encrypted:
const encrypted = encrypt(value, KEY)
// Decrypt only when needed
```

7.4 Input Validation

```
// Validate all user inputs before using
const email = req.body.email?.trim()
if (!email || !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) {
  return res.status(400).json({ error: 'Invalid email' })
}

// Parameterized queries prevent SQL injection:
const result = await pool.query(
  'SELECT * FROM users WHERE email = $1',
  [email]  // Parameterized, not string concatenation
)
```

---

8. PERFORMANCE OPTIMIZATION

8.1 Database Optimization

Indexes:

- Email, mobile (lead lookup)
- Owner, stage (filtering)
- created_at (sorting)
- utr_number (payment lookup)

Query Optimization:

- Use EXPLAIN ANALYZE to identify slow queries
- Avoid N+1 queries (use JOINs)
- Limit SELECT to needed columns (not SELECT *)
- Use pagination (LIMIT/OFFSET)

Connection Pooling:

```
const pool = new Pool({
  connectionString: DATABASE_URL,
  max: 20,  // Max connections
  idleTimeoutMillis: 30000,
  connectionTimeoutMillis: 2000,
})
```

8.2 API Optimization

Response Caching:

- Cache integration settings (5 min TTL)
- Cache user roles (session duration)
- Don't cache: real-time lead counts, payments

Payload Optimization:

- Remove lead_details JSONB from list views (72MB → 2MB)
- Only send needed fields
- Compress JSON responses (gzip)

Rate Limiting:

```
const rateLimit = require('express-rate-limit')
app.use(rateLimit({
  windowMs: 15 * 60 * 1000,  // 15 minutes
  max: 100  // max 100 requests per IP
}))
```

8.3 Frontend Optimization

Code Splitting:

- Lazy load pages using React.lazy()
- Load integrations only when opened

Caching:

- Cache static assets (JS, CSS) with hash in filename
- Browser caches for 1 year
- Cache-busting on deploy

Rendering:

- Use React.memo() for expensive components
- Avoid unnecessary re-renders
- Use key prop in lists

---

9. TESTING STRATEGY

9.1 Unit Testing

```
// Test individual functions
describe('passwordValidation', () => {
  it('should require minimum 8 characters', () => {
    expect(isValidPassword('Pass123')).toBe(false)
    expect(isValidPassword('Password123')).toBe(true)
  })
})
```

9.2 Integration Testing

```
// Test API endpoints
describe('POST /api/leads', () => {
  it('should create a new lead', async () => {
    const res = await request(app)
      .post('/api/leads')
      .set('Authorization', `Bearer ${token}`)
      .send({ name: 'Rahul', email: 'rahul@example.com' })

    expect(res.status).toBe(201)
    expect(res.body.data.id).toBeDefined()
  })
})
```

9.3 Performance Testing

```
# Load test with 100 concurrent users
ab -n 10000 -c 100 https://crm.cutmap.ac.in/api/leads?page=1

# Expected: <500ms p95 latency, <2% error rate
```

9.4 Security Testing

- SQL Injection: Try email: "'; DROP TABLE users; --"
- XSS: Try name: "<script>alert('xss')</script>"
- CSRF: Verify same-site cookie flag
- Penetration testing by external firm

---

10. DEPLOYMENT ARCHITECTURE

10.1 Deployment Environment

```
Production Server (Linux)
├── Node.js process (ccrm-backend systemd service)
├── PostgreSQL database
├── /var/www/ccrm/
│   ├── server/  (backend code)
│   ├── ccrm/    (frontend build)
│   └── uploads/ (uploaded files)
└── Reverse proxy (Nginx)
    └── HTTPS termination
```

10.2 Deployment Process

```
1. Developer commits to GitHub
2. CI/CD pipeline (if enabled):
   - Run tests
   - Build frontend
   - Deploy to production
3. Manual deployment (current):
   - Run DEPLOY.sh on server
   - Copy files to /var/www/ccrm/
   - Restart ccrm-backend service
```

10.3 Monitoring & Logging

Application Logs:

```
sudo journalctl -u ccrm-backend -f
```

Key Metrics:

- Response time (p50, p95, p99)
- Error rate
- Database query time
- Cron job execution status

---

11. SCALABILITY CONSIDERATIONS

11.1 Current Architecture

- **Single Server:** Node.js + PostgreSQL on one machine
- **Leads:** 200,000+
- **Concurrent Users:** 100+
- **Storage:** Local filesystem + S3

11.2 Future Scalability (Phase 2)

Horizontal Scaling:

- Load balancer (Nginx, HAProxy)
- Multiple Node.js instances
- Connection pooling to PostgreSQL

Database Scaling:

- Read replicas for analytics
- Sharding by campus (if >1M leads)
- Archive old leads to cold storage

Caching Layer:

- Redis for session storage
- Cache integration settings
- Cache frequently accessed data

---

12. DISASTER RECOVERY

12.1 Backup Strategy

Daily Backups (3:00 AM IST):

- Database: Full PostgreSQL dump (gzipped)
- Files: uploads/ directory (tarred)
- Logs: Last 24 hours

Storage: AWS S3 (replicated across regions)

Retention: 30 days minimum (configurable via S3 lifecycle policies)

12.2 Recovery Procedures

Database Recovery:

```
aws s3 cp s3://bucket/backups/2026-06-02/db.sql.gz .
gunzip db.sql.gz
psql -U ccrm_user ccrm_db < db.sql
```

Files Recovery:

```
aws s3 cp s3://bucket/backups/2026-06-02/uploads.tar.gz .
tar -xzf uploads.tar.gz -C /var/www/ccrm/
```

RTO (Recovery Time Objective): <4 hours
RPO (Recovery Point Objective): <24 hours

---

13. DOCUMENT CHANGE HISTORY

Version	Date	Author	Changes
1.0	Jan 2026	Team	Initial design
1.1	Feb 2026	Team	Added payment verification
1.2	Mar 2026	Team	Optimized database queries
1.3	Apr 2026	Team	Added cron job architecture
1.4	May 2026	Team	Refined API design
1.5	Jun 2026	Team	Final version with S3 backup

End of SDD Document